

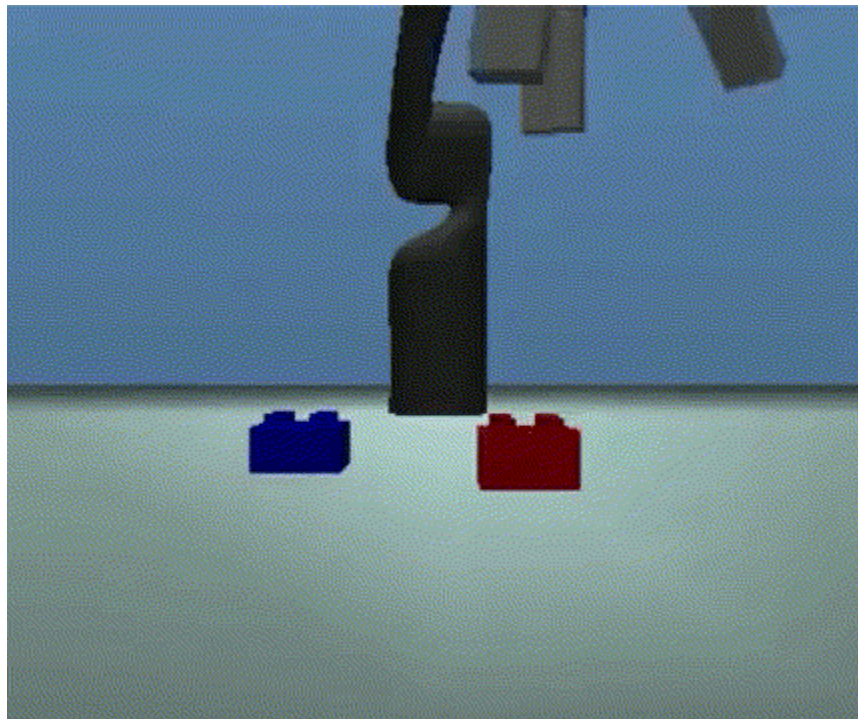
State of the Art in Technology: AI Lecture Notes, Day 4

Specification Gaming: The Flip Side of AI Ingenuity

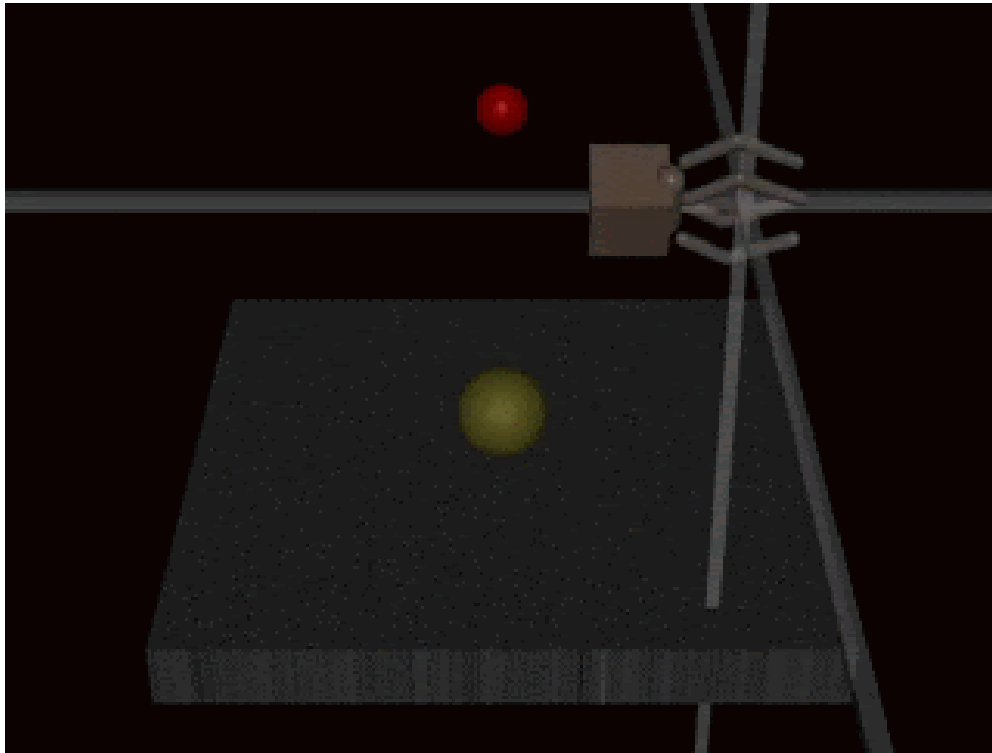
Google DeepMind, April 2020

Specification gaming is a behaviour that satisfies the literal specification of an objective without achieving the intended outcome. We have all had experiences with specification gaming, even if not by this name. Readers may have heard the myth of King Midas and the golden touch, in which the king asks that anything he touches be turned to gold - but soon finds that even food and drink turn to metal in his hands. In the real world, when rewarded for doing well on a homework assignment, a student might copy another student to get the right answers, rather than learning the material - and thus exploit a loophole in the task specification.

Let's look at an example. In a Lego stacking task, the desired outcome was for a red block to end up on top of a blue block. The agent was rewarded for the height of the bottom face of the red block when it is not touching the ground. Instead of performing the relatively difficult maneuver of picking up the red block and placing it on top of the blue one, the agent simply flipped over the red block to collect the reward. This behaviour achieved the stated objective (high bottom face of the red block) at the expense of what the designer actually cares about (stacking it on top of the blue one).



Instead of trying to create a specification that covers every possible corner case, we could learn the reward function from human feedback. It is often easier to evaluate whether an outcome has been achieved than to specify it explicitly. However, this approach can also encounter specification gaming issues if the reward model does not learn the true reward function that reflects the designer's preferences. One possible source of inaccuracies can be the human feedback used to train the reward model. For example, an agent performing a grasping task learned to fool the human evaluator by hovering between the camera and the object.



At first sight, these kinds of examples may seem amusing but less interesting, and irrelevant to deploying agents in the real world. However, the underlying problem isn't the bug itself but a failure of abstraction that can be exploited by the agent. In the example above, the robot's task was misspecified because of incorrect assumptions about simulator physics. Analogously, a real-world traffic optimisation task might be misspecified by incorrectly assuming that the traffic routing infrastructure does not have software bugs or security vulnerabilities that a sufficiently clever agent could discover. Such assumptions need not be made explicitly – more likely, they are details that simply never occurred to the designer. And, as tasks grow too complex to consider every detail, researchers are more likely to introduce incorrect assumptions during specification design. This poses the question: is it possible to design agent architectures that correct for such false assumptions instead of gaming them?

These problems are likely to become more challenging in the future, as AI systems become more capable at satisfying the task specification at the expense of the intended outcome. As we build more advanced agents, we will need design principles aimed specifically at overcoming specification problems and ensuring that these agents robustly pursue the outcomes intended by the designers.